Training Report

on

Java Full Stack Development

At

Defence Research and Development Laboratory (DRDL)

Kanchanbagh,Hyderabad – 500058.

In Partial Fullfillment of the Diploma

Diploma

In

Computer Science And Engineering (CSE)

TRR College of Technology (Polytechnic)

Submitted by

Palarapu Hansika

CSE 3rd YEAR

24124-CS-054

Under The Guidelines of

Sri. B.V. Hari Krishna Nanda, Scientist 'G'

Technology Director,DIT,DRDL

DECLARATION

I, P.Hansika, hereby declare that this Report is being submitted in Partial Fullfillment the Requirements of my Internship at Defence Research and Development Organisation(DRDL).

This Report is an original work carried out by me during my Internship and has been completed under the Guidance of **Sri B.V. Hari Krishna Nanda, Scientist 'G'** and **Smt. B. Deepika,Scientist 'E', DRDL**. This internship was Undertaken as a part of the Academic Curriculam of TRR College of Technology(Polytechnic),Meerpet,Hyderabad.

Signature

P. Hansika

Signature

Smt. B. Deepika, Scientist 'E'

Head, SDD, Hyderabad.

Signature

Sri. B.V. Hari Krishna Nanda, Scientist 'G'

Technology Director, DIT, DRDL, Hyderabad.

ACKNOWLEDGEMENT

I would like to express My Sincere gratitude to **Sri. B.V. Hari Krishna Nanda (Scientist 'G')**, Defence Research and Development Laboratory(DRDL), Hyderabad, for providing me with the opportunity to Undertake this Internship and work on a practically relevant and technically enriching project.

I am deeply grateful to My Guide, **Smt. B. Deepika (Scientist 'E')**, Defence Research and Development Laboratory(DRDL), for their invaluable guidance, constant encouragement and constructive feedback throughout the course of this project their expertise and patient mentoring were instrumental in shaping the direction and quality of this work and helping me successfully complete My Project on Java Full Stack.

I also wish to Acknowledge the technical and administrative staff of DRDL for their support and for providing access to the necessary resources and infrastructure.

HISTORY

The Defence Research and Development Laboratory (DRDL) was established in 1961 under the Defence Research and Development Organisation (DRDO) objective of developing indigenous Missile Systems for India's Defence Forces Located at Kanchanbagh, Hyderabad, DRDL marked the beginning of India's journey towards Self-Reliance in Missile technology during an era when the Country was largely dependent on foreign defence equipment.

In its formative years, DRDL initiated Several Pivotal Programmes such as Project Indigo, a surface-to-air Missile Project which, although not fully successful, laid the groundwork for more Advanced initiatives. This was followed by Project Devil and Project Valiant in the 1970s, which contributed significantly to the development of Short-Range Missile technologies and laid a strong foundation for future Projects.

The Laboratory's efforts culminated in the launch of the Integrated Guided Missile Development Programme (IGMDP) in the 1980s, a Major Milestone in India's Missile development history Defence Research and Development Laboratory played a critical role in the successful design and development of key strategic Missile Systems under this programme, including Prithvi, Agni, Akash and Nag.

Over the years, Defence Research and Development Laboratory has evolved into a Premier Centre of excellence in Missile technology, specialising in areas such as aerodynamics, propulsion, guidance and control, warhead technology and missile systems integration with state-of-the-art infrastructure for design, simulation, testing and validation, Defence Research and Development Laboratory supports end-to-end development of both tactical and strategic missile systems.

Today, Defence Research and Development Laboratory continues to drive innovation by contribution to advanced to next generation missile systems such as hypersonic, weapons, interceptor missiles and long-range strategic deterrents In alignment with DRDO's vision of Self-Reliance in defence technologies, Defence Research and Development Laboratory actively collaborates with other DRDO laboratories, academic institutions and Indian industries Its continued commitment to technological excellence plays a vital role in enhancing India's defence preparedness and securing its strategic autonomy.

Projects

1. User Management System
    . User Registration
    . Login
    . Change Password
    . Forget Password
2. HS Mail Hub

1. User Management System

INTRODUCTION

User Management System (UMS) is an application module designed to handle user lifecycles within a software ecosystem. It securely registers users, authenticates credentials, enforces password security policies, handles account recovery, and maintains active session states.

In modern web applications, a robust UMS ensures that sensitive user data (like passwords and personal info) is securely stored, validated, and managed to prevent unauthorized access.

Software's Installed

To build and run this full-stack User Management System, the following software, tools, and libraries are utilized across the development environment:

Frontend Environment:

- **React.js:** A JavaScript library used for building user interfaces with a single-page application (SPA) architecture.
- **Axios:** A promise-based HTTP client used to send asynchronous REST requests from React to the Spring Boot backend.
- **Node.js & npm:** JavaScript runtime and package manager used to install dependencies and run the React development server (localhost:3000).

Backend Environment:

- **Java Development Kit (JDK):** Provides the runtime environment and compiler for executing Java code.
- **Spring Boot:** An open-source Java-based framework used to build standalone, production-grade Spring applications and REST APIs quickly.
- **Spring Data JPA / Hibernate:** An ORM (Object-Relational Mapping) framework that simplifies database interactions by mapping Java entities to database tables.
- **Maven Wrapper (mvnw.cmd):** A build automation tool included locally in the project to manage dependencies, build the project package, and run the backend server without requiring a global Maven installation.

Database & Management:

- **Oracle Database (orcl SID):** An enterprise relational database management system used to persistently store user credentials, security questions, and profile metadata.
- **Oracle JDBC Driver:** The database driver enabling Java application connectivity with the Oracle database using SCOTT/tiger credentials.

Development Tools:

- **Visual Studio Code (VS Code):** The primary code editor hosting both frontend and backend source workspaces.
- **PowerShell / Terminal:** Used for executing build scripts, managing server processes, and starting both development environments.

Execution Process

1.Start the Backend Server:

- Open a terminal window in your backend directory (/Backend).
- Run the Maven wrapper command to compile and launch the application on port 8080

VS Terminal/Cmd:

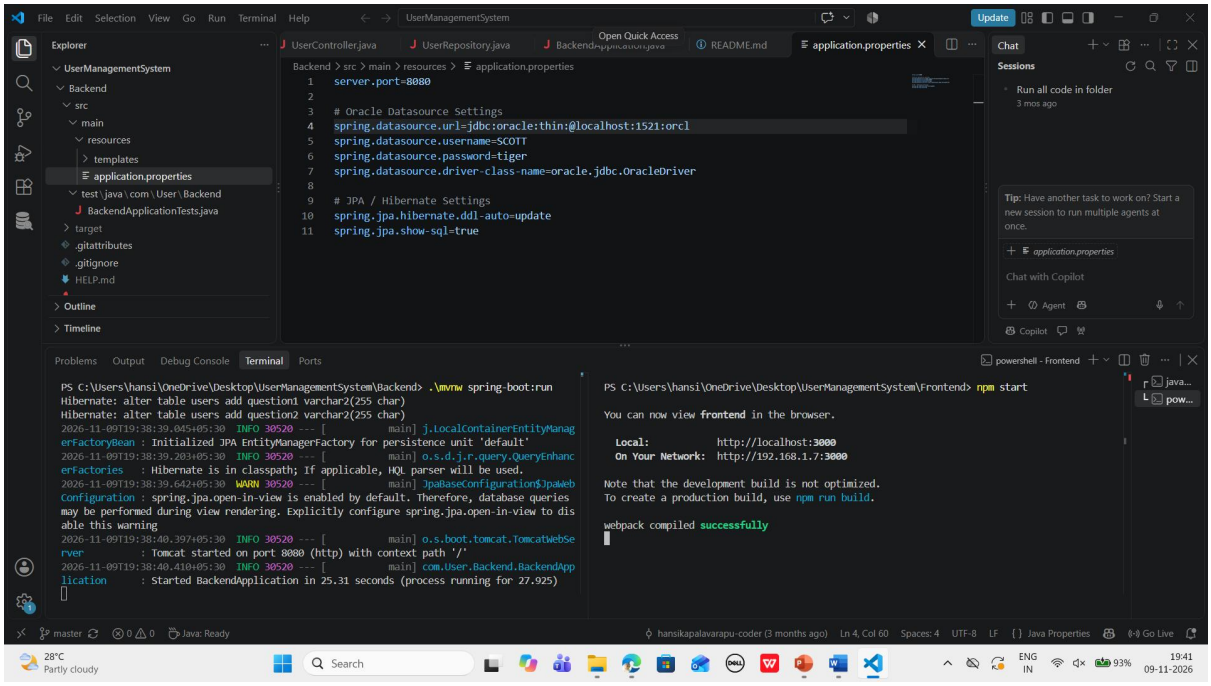
.\mvnw spring-boot:run

2.Start the Frontend Development Server:

- Open a separate terminal window in your frontend directory (/frontend).
- Start the React client application

VS Terminal/Cmd:

npm start



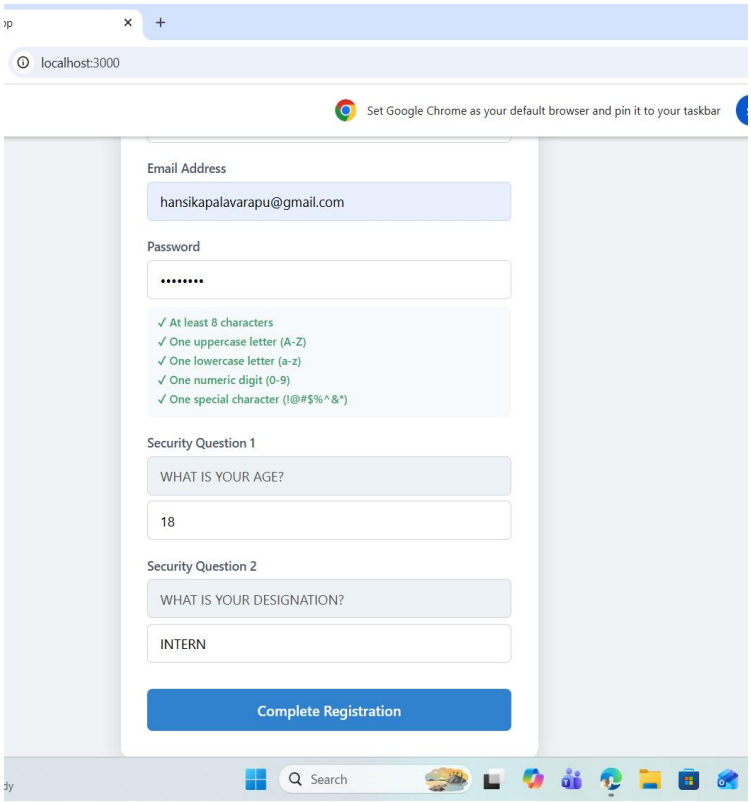
. User Registration

. Purpose:

The entry point for new users to create an account within the system securely, capturing essential user details and setting up account credentials.

. Functionality:

- *Collects user input including Full Name, Email Address, Date of Birth, Password, and answers to two security questions.*
- *Implements real-time visual validation rules for passwords, ensuring minimum length (8+ characters), uppercase and lowercase letters, numeric digits, and special characters before enabling the registration button.*
- *Communicates via an HTTP POST request to /api/register. The backend checks if the email already exists in the Oracle database; if unique, it hashes or stores the password, records the registration timestamp(passwordUpdatedAt), sets account status to active, and saves the entity.*



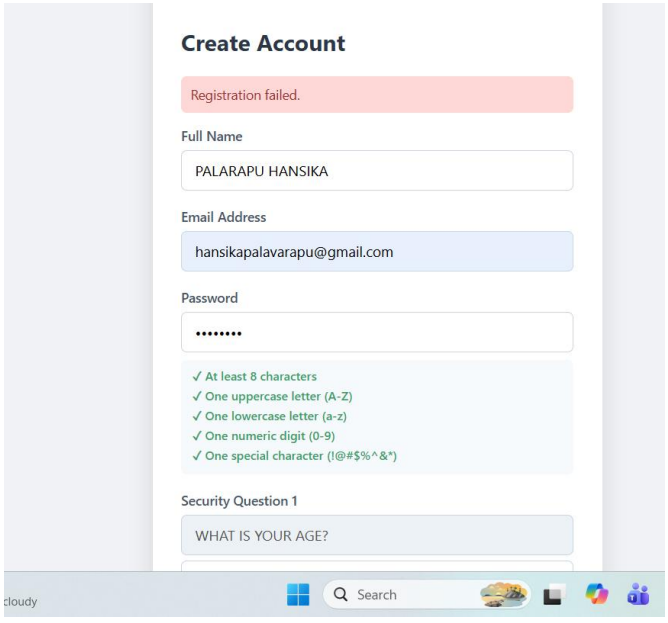
. Login

. Purpose:

Authenticates registered users to grant secure access to protected application views.

. Functionality:

- *Validates email and password payloads via an HTTP POST request to /api/login.*
- *Incorporates a built-in password expiration policy (checking if the password was last updated over 30 days ago). Expired credentials trigger a 403 Forbidden response status, alerting the user and automatically redirecting them to the recovery module.*
- *Returns user session metadata upon successful authentication to initialize the user dashboard.*



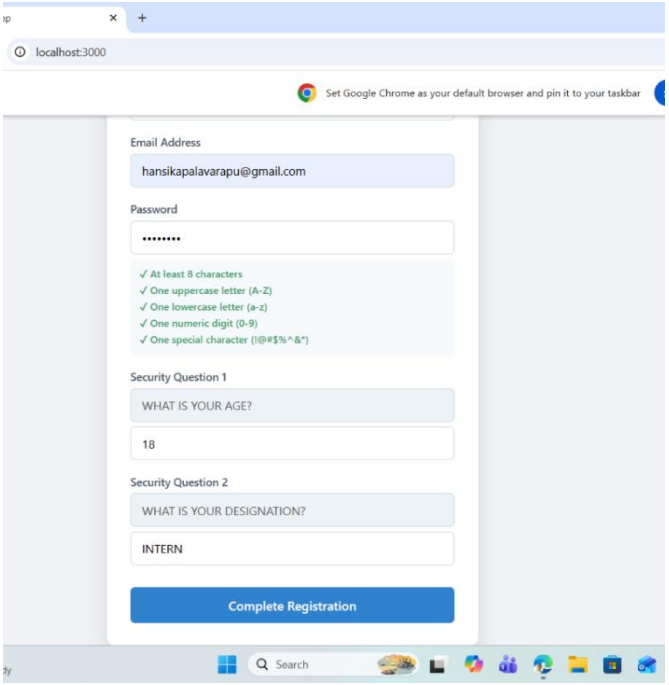
. Change Password

. Purpose:

Offers a self-service recovery mechanism for users who forgot their credentials or whose passwords have expired, avoiding manual intervention.

. Functionality:

- **Step 1:** The user provides their registered email to retrieve their specific pre-set security questions (/api/forgot-password/get-questions).
- **Step 2:** The user answers both security questions and submits a new password meeting security rules (/api/forgot-password/reset).
- The backend validates case-insensitive security answers, updates the password hash/value, refreshes the update timestamp, and saves changes.



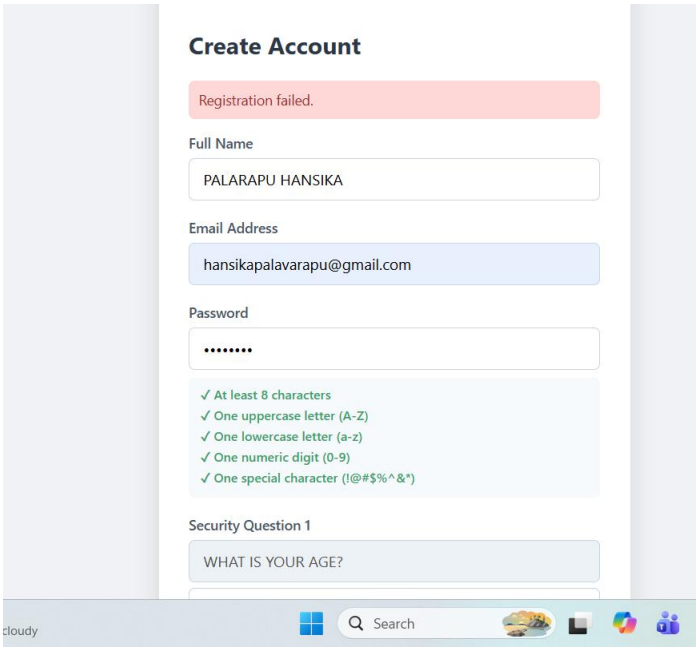
. Forget Password

. Purpose:

A secure control panel and landing page visible exclusively to authenticated users.

. Functionality:

- Renders session metadata such as Full Name, Email Address, Date of Birth, and Account Status.
- Houses a **Log Out** action handler that clears user session state (currentUser set to null) and safely routes the application view back to the login module.



2 . HS Mail Hub

INTRODUCTION

HS MailHub is a full-stack enterprise communication and management web application designed for **HS Private Limited**. The platform provides a unified, role-based dashboard for employees and managers to interact seamlessly.

Instead of relying on third-party live email infrastructure—which introduces external security risks, complex authentication requirements, rate limiting, and network latency—HS MailHub utilizes an internal simulated messaging architecture. This ensures high system availability, instant local message delivery during testing and live demonstrations, and zero risk of exposing sensitive email credentials or API keys.

SOFTWARE'S INSTALLED

Backend Environment

- **Java 21**: Core programming language utilizing modern language features and runtime optimizations.
- **Spring Boot**: Web application framework for creating RESTful API endpoints, dependency injection, and application lifecycle management.
- **Spring Data JPA / Hibernate**: Object-Relational Mapping (ORM) framework to streamline backend database operations.
- **Maven Wrapper (mvnw)**: Build automation tool ensuring consistent dependency management across development environments.

Frontend Environment

- **React**: JavaScript library for building component-based, single-page application (SPA) user interfaces.
- **Vite**: Modern frontend build tool providing fast module replacement (HMR) and optimized build bundles.
- **JavaScript / HTML5 / CSS3**: Core Web standards for layout structure, styling, and interactive client-side logic.

Database & Version Control

- **MySQL**: Relational Database Management System (RDBMS) for persistent storage of user profiles, messages, and role assignments.
- **Git & GitHub**: Distributed version control system and remote repository hosting (hansikapalarapu-coder/HS-Pvt-Ltd).

Development Tools:

  - **Visual Studio Code (VS Code):** The primary code editor hosting both frontend and backend source workspaces.
  - **PowerShell / Terminal:** Used for executing build scripts, managing server processes, and starting both development environments.

Execution Process

1.Start the Backend Server:

- Open a terminal window in your backend directory (/Backend).
- Run the Maven wrapper command to compile and launch the application on port 8080

VS Terminal/Cmd:

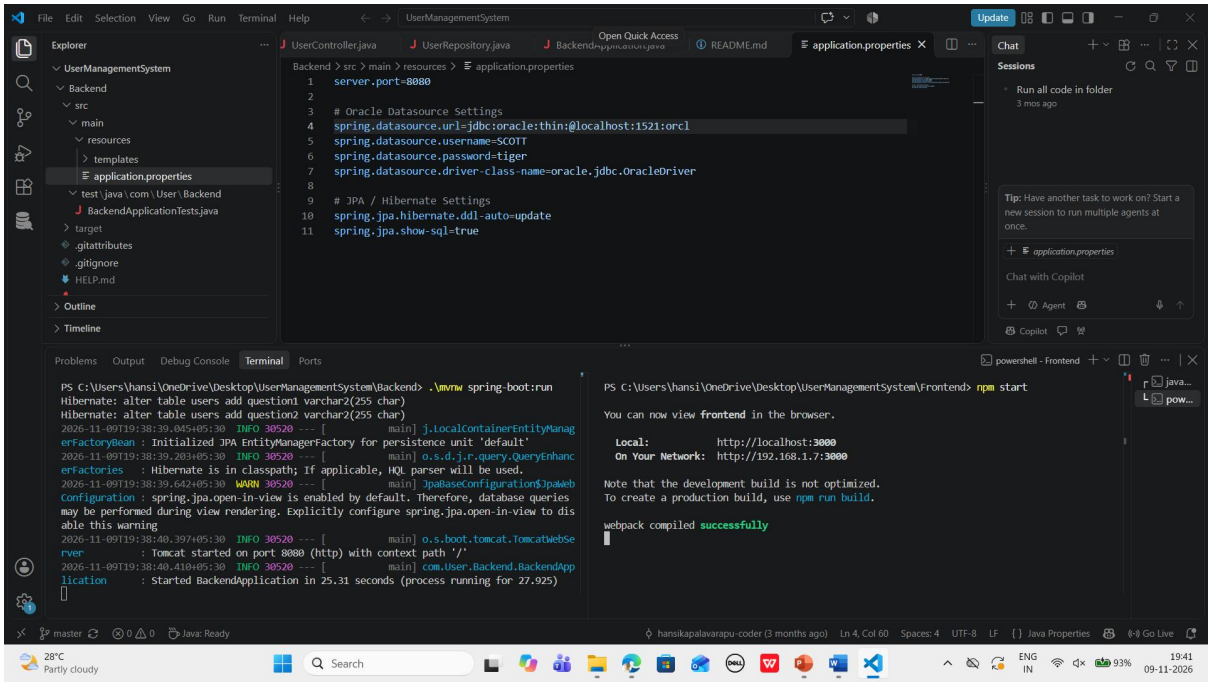
.\mvnw spring-boot:run

2.Start the Frontend Development Server:

- Open a separate terminal window in your frontend directory (/frontend).
- Start the React client application

VS Terminal/Cmd:

npm start



Technical Disadvantages of Live Email Integration

The project intentionally bypasses live external SMTP email delivery in favor of a simulated messaging system. The primary technical disadvantages, security vulnerabilities, and operational drawbacks of live email integration include:

1. Security & Credential Protection Risks

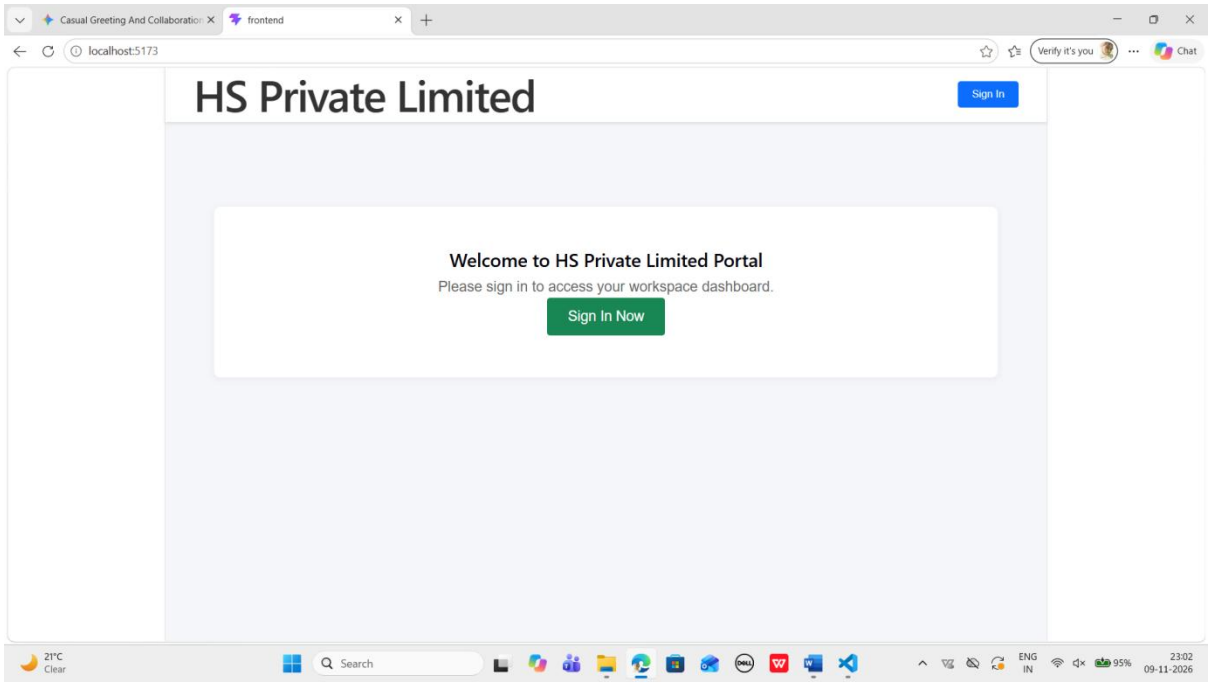
- **Credential Exposure**: Hardcoding or storing live SMTP passwords and private OAuth client secrets in configuration files risks leaking sensitive corporate or personal accounts if pushed to public source control repositories.
- **Authentication Overhead**: Modern email services (Gmail, Outlook) enforce App Passwords, Multi-Factor Authentication (MFA) exceptions, and complex OAuth2 token-handling workflows, adding unnecessary operational friction.

2. Provider Rate Limits & Delivery Throttling

- **Quota Restrictions**: Free and developer-tier SMTP servers impose strict hourly and daily sending limits, triggering account suspensions during continuous testing.
- **Anti-Spam Blocklisting**: Automated emails generated locally (localhost) lacking domain authentication records (SPF, DKIM, DMARC) are frequently marked as spam or dropped entirely by recipient mail servers.

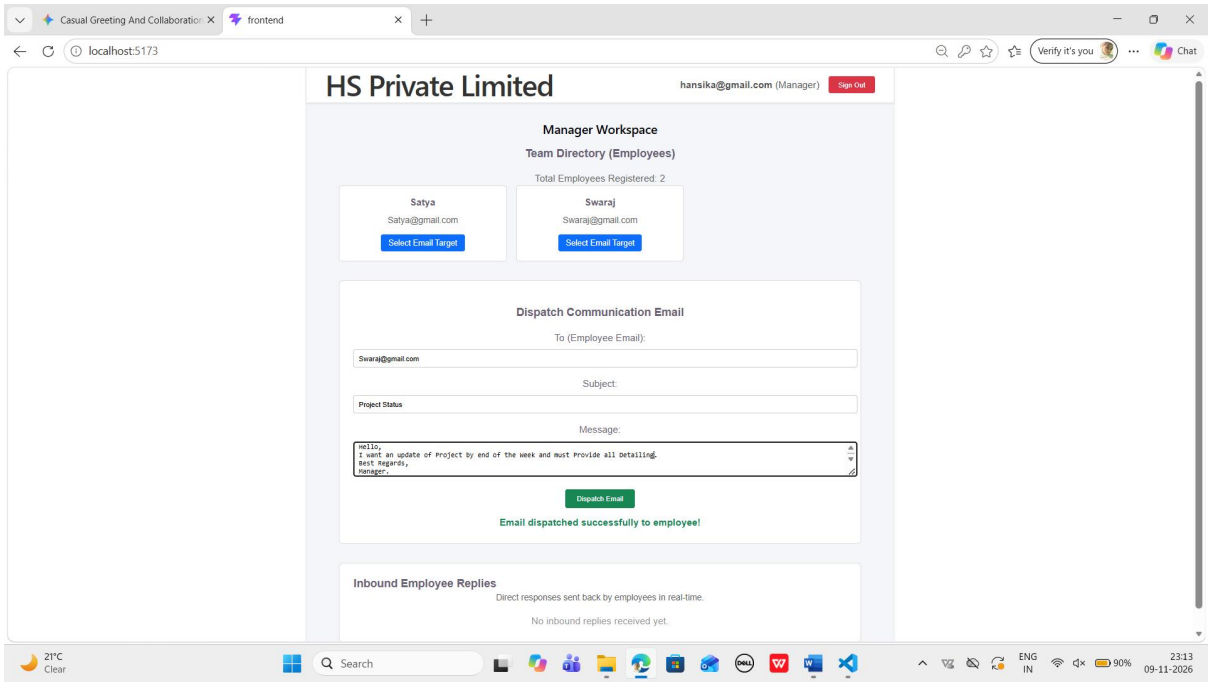
3. Latency, Network Dependency & Presentation Vulnerability

- **Network Overhead**: Live SMTP handshakes introduce network latency, disrupting the instantaneous responsiveness of the web interface during user evaluations.
- **Single Point of Failure**: System operations become strictly dependent on active internet connectivity and open outbound SMTP ports (25, 465, 587), making live presentations vulnerable to firewall blocks or Wi-Fi drops.
- **Inbox Pollution**: Real-world delivery clutters physical inboxes with test artifacts and dummy data during development iterations.



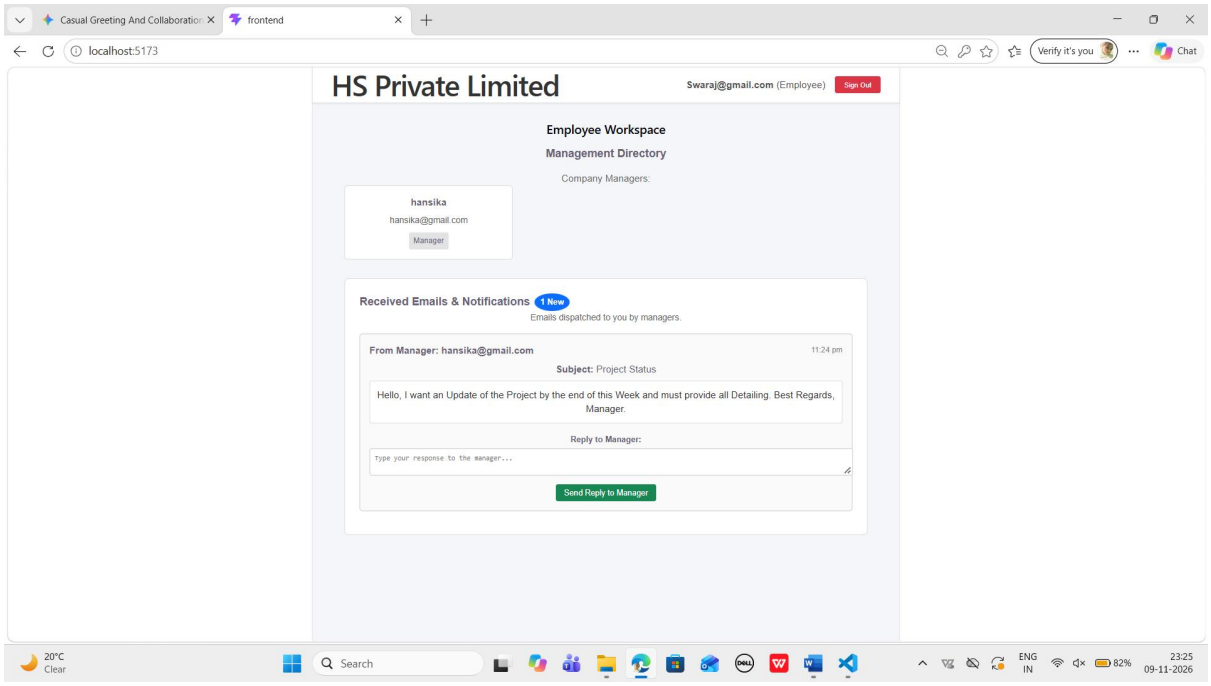
Manager Login and Email Dispatch to Employee

(Send Email to Employer)



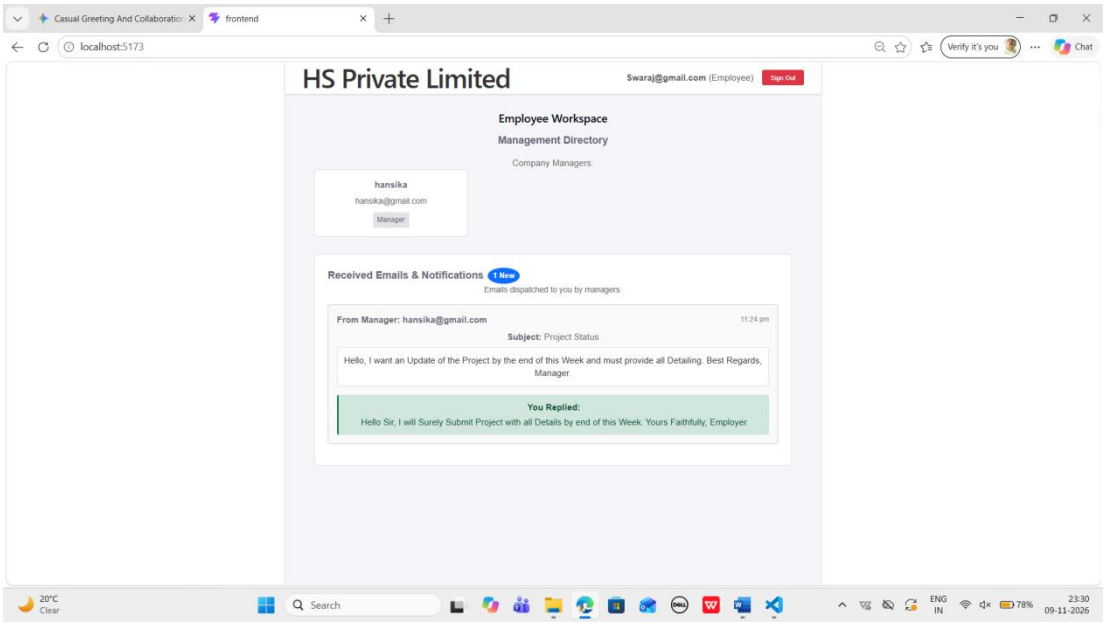
Employer Login and Email Dispatch to Manager

(Receive the Email from Manager)



Employer Replying the Email by Manager

(Send Reply to Email Sent by Manager)



Manager Receiving the Reply Email from Employer

(Received Email by Employer)

